

Reporte técnico de funcionamiento Frontend

El frontend cubre las operaciones CRUD para las 4 entidades principales que son los administradores, gerentes, usuarios, dispositivos y la autenticación con JWT.

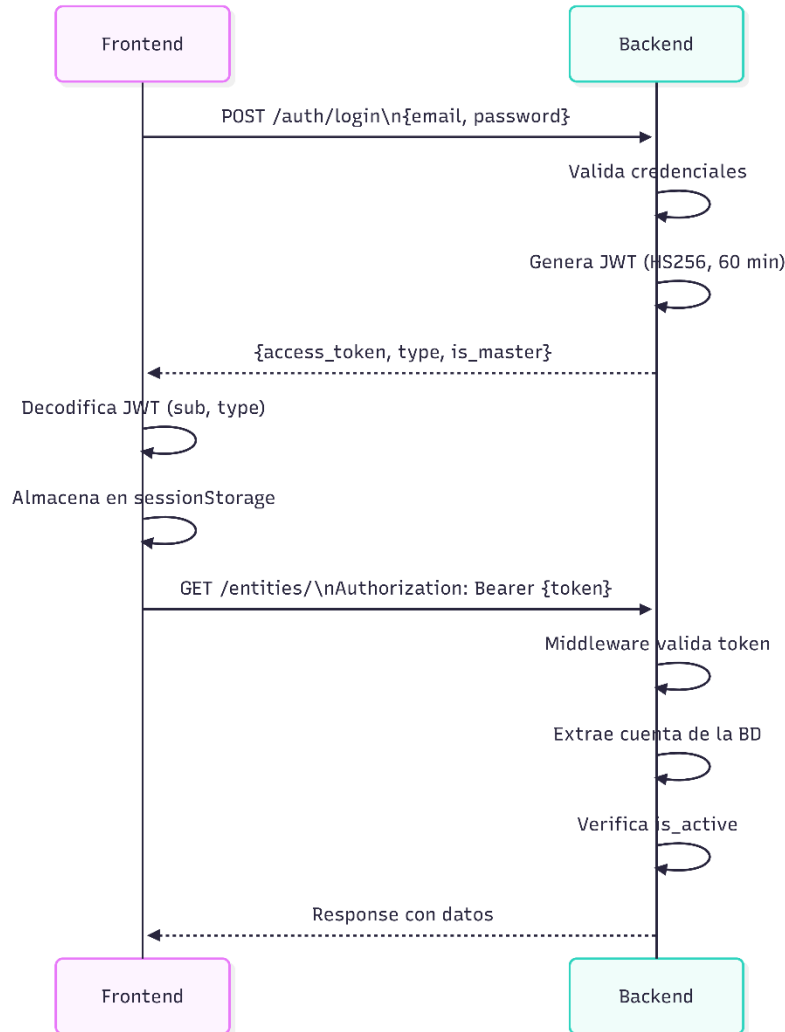
Todas estas funcionalidades están probadas en un entorno Docker, sin embargo, el sistema de servicios, roles, permisos, aplicaciones y tickets cuyas tablas existen en la base de datos, no tienen endpoints en el backend por lo tanto no hay interfaz en el frontend para su manipulación.

Funcionalidades

- Autenticación y Autorización

Funcionalidad	Estado	Detalles
Login con email y contraseña	Funcional	POST /api/v1/auth/login → JWT token
Duración de sesión	Funcional	Token almacenado en sessionStorage, hace la validación de expiración al cargar
Protección de rutas	Funcional	ProtectedRoute verifica accountType y isMaster
Cierre de sesión	Funcional	Limpia sessionStorage y redirige a login
Diferencia de roles	Funcional	Admin Master / Admin Normal / Gerente / Usuario

A continuación se detalla en un diagrama de comunicación el proceso de la autenticación de usuarios para mejor explicación:



- **CRUD de administradores**

Operación	Endpoint	Método	Estado
Listar	/api/v1/administrators/	GET	Correcto
Crear	/api/v1/administrators/	POST	Correcto
Editar	/api/v1/administrators/{id}	PATCH	Correcto
Eliminar	/api/v1/administrators/{id}	DELETE	Correcto

- **CRUD de gerentes**

Operación	Endpoint	Método	Estado
Listar	/api/v1/managers/	GET	Correcto
Crear	/api/v1/managers/	POST	Correcto

Editar	/api/v1/managers/{id}	PATCH	Correcto
Eliminar	/api/v1/managers/{id}	DELETE	Correcto

- **CRUD de usuarios**

Operación	Endpoint	Método	Estado
Listar	/api/v1/users/	GET	Correcto
Crear	/api/v1/users/	POST	Correcto
Editar	/api/v1/users/{id}	PATCH	Correcto
Eliminar	/api/v1/users/{id}	DELETE	Correcto

- **CRUD de dispositivos**

Operación	Endpoint	Método	Estado
Listar	/api/v1/devices/	GET	Correcto
Crear	/api/v1/devices/	POST	Correcto
Editar	/api/v1/devices/{id}	PATCH	Correcto
Eliminar	/api/v1/devices/{id}	DELETE	Correcto

Nota de seguridad:

- El backend no tiene guards de autorización en el controlador de gerentes ni en el de dispositivos, el frontend restringe la UI solo a administradores, pero la API es accesible para cualquier usuario autenticado.

Funcionalidades no implementadas

- **Módulo de servicios**

Las tablas existen en la BD pero no hay endpoints en el backend para hacer conexión al UI en el frontend.

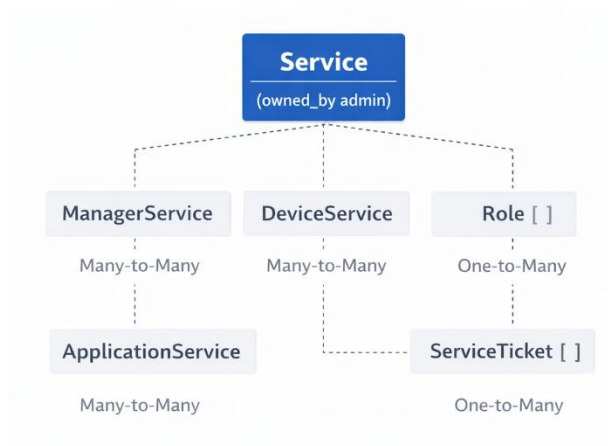


Ilustración 1 Tablas y sus relaciones

Se sugieren los siguientes endpoints para el correcto manejo y manipulación de los servicios:

- /api/v1/services/
- /api/v1/services/{id}/managers/
- /api/v1/services/{id}/devices/
- /api/v1/services/{id}/roles/

¿Por qué estos endpoints?

Los proponemos basados en los recursos REST, donde es importante que cada entidad se exponga como un recurso, y se hace la separación de responsabilidades:

Acción	Endpoint
Crear servicio	/services
Asignar manager	/services/{id}/managers
Asignar device	/services/{id}/devices
Gestionar roles	/services/{id}/roles

- **Metodos para los endpoints:**

Acción	Método HTTP	Ejemplo
Listar servicios	GET	/services
Obtener uno	GET	/services/{id}
Crear	POST	/services
Actualizar	PATCH	/services/{id}
Eliminar	DELETE	/services/{id}

- **Módulo de Roles y Permisos**

Existen los siguientes modelos definidos: `Role`, `RolePermission`, `UserRole` pero sin endpoints por lo tanto tampoco uso en el UI

Lo que falta:

- CRUD de roles para crear/editar/eliminar roles dentro de un servicio
- Permisos por rol
- Asignación usuario-rol para vincular usuarios a roles específicos
- **No hay endpoints para uso de tickets**

Se requieren endpoints para poder crear, editar o eliminar tickets.